Efficient Algorithms for Finding a Heaviest Common Increasing Subsequence

Ren-Fu Wang[†], Zi-Jien Ho[†], Wen-Ting Fan[†], and Yao-Ting Huang[†‡]
[†]Department of Computer Science and Information Engineering
National Chung-Cheng University, Taiwan.
[‡]Corresponding Author: ythuang@cs.ccu.edu.tw

Abstract

This paper studies several generalized versions of the longest common increasing subsequence (LCIS) problem. Given two number sequences (A and B) and a weight function for each pair of matched numbers in A and B, the heaviest common increasing subsequence (HCIS) problem asks for a common increasing subsequence of A and B with maximum total weight. For two sequences of lengths m and n, we propose an improved algorithm with $O(mn)$-running time. For the weakly increasing version of the HCIS problem (HCWIS), we develop an $O(mn)$-time algorithm, which is also the first general solution to the unweighted LCWIS problem. For the HCIS problem of k sequences (k-HCIS) where $k \geq 3$, we present an algorithm with $O(kn^k)$-running time, which is faster than previous results for the LCIS problem of k sequences. Compared with previous approaches, our algorithms improve efficiency without replying on any sophisticated data structure.

1 Introduction

In computational biology, the comparison of genomes of two species can uncover their conserved genomic regions or lineage-specific differences [15]. However, the direct comparison of two entire genomes is time-consuming and infeasible. Therefore, existing approaches often start by finding genomic segments of high similarity in both genomes (termed synteny blocks) [16]. These blocks are usually formulated into a permutation of numbers in each genome. Then, a largest subset of blocks that appear in the same order in both genomes is identified. This is often done by solving the problems of longest common subsequence (LCS) or longest increasing subsequence (LIS) [8]. With the consideration of distinct similarity of each block, Chan *et al.* [4] assigned different weights to blocks and obtained better accuracy be solving Heaviest Increasing Subsequence (HIS) problem.

The LCS and LIS problems have been extensively studied in computer science [7, 19]. The LCS problem can be solved by a classical dynamic programming approach in $O(mn)$ time, where m and n are the lengths of two sequences. Masek and Paterson [14] gave a faster algorithm with $O(mn/\log n)$ time, assuming the sizes of subsequences are bounded. The LIS problem was independently solved by Schensted [17] and Knuth [12] in $O(n \log n)$ time. Suppose that the input sequence is a permutation of {1, 2, ..., n}. Bespamyatnikh and Segal [2] gave a faster algorithm that runs in $O(n \log \log n)$ time. On the other hand, the weighted versions of LCS and LIS problems, respectively termed heaviest common subsequence (HCS) and HIS problems, were solved by Jacobson and Vo [11] in $O(n \log n)$ time. A review report of these results can be found in [1].

In recent years, Yang *et al.* [20] proposed a new formulation called longest common increasing subsequence (LCIS) for comparing genomes of three species. Given two permutated number sequences, LCIS is a common increasing subsequence of both sequences with maximum length. An O(mn)-time and O(mn)-space algorithm was developed for finding the LCIS of two sequences [20]. Subsequently, Chan *et al.* [5] developed a new algorithm with $O(\min(r \log L, nL + r) \log \log n + Sort(n))$ time, where n is the length of each sequence, r is the number of pairs of positions at which the two sequences match, and L is the length of the LCIS. Nevertheless, the number of matches r is $\Omega(mn)$ in the worst case. Recently, Brodal *et al.* [3] described an algorithm with $O((m + nL) \log \log \alpha + Sort(n))$ time, where α is the size of the alphabet. However, L can be still up to $O(n)$ in the worst case. When $\alpha = 3$, Brodal *et al.* [3] further

solved the weakly increasing version of the LCIS problem (termed LCWIS) in $O(m+n\log n)$ time, which allows a monotonically increasing (rather than strictly increasing) solution. For the LCIS problem of k sequences where $k \geq 3$ (called k-LCIS), Brodal *et al.* [3] and Chan *et al.* [5] presented $O(\min(kr^2, r\log^{k-1} r \log\log r))$- and $O(\min(r\log L\log^k r) + k * Sort(n))$-time algorithms, respectively, where r, k, L, and n are as mentioned above. However, the efficiency of these approaches all rely on existing sophisticated data structure.

In this paper, we studied the weighted versions of the LCIS , LCWIS, and k-LCIS problems without relying on any sophisticated data structure. Given two sequences and a weight function for each pair of matched numbers, the Heaviest Common Increasing Subsequence (HCIS) problem asks for a common increasing subsequence of both sequences with maximum total weight. The LCIS problem is a special case of the HCIS problem when setting unit weight for each pair of matched numbers. One straightforward approach for solving HCIS would take $O(n^3)$ time. Recently, Sung [18] proposed an $O(mn\log n)$-time algorithm. This paper presents a faster algorithm with $O(mn)$ time. For the weakly increasing version of the HCIS problem (called HCWIS), we propose an $O(mn)$-time algorithm, which is also the first general solution to the LCWIS problem. For the HCIS problem with k sequences (called k-HCIS), we describe an $O(kn^k)$-time algorithm, which runs faster than existing algorithms for the k-LCIS problem. Table 1 summarizes the previous results and results in this paper. Note that LCIS, LCWIS, and k-LCIS are special cases of the HCIS, HCWIS, and k-HCIS problems, respectively.

When comparing genomes of three or more species, algorithms for LCIS, LCWIS, and k-LCIS can be used to identify a largest subset of synteny blocks that appear in the same order in all genomes. However, the lack of considering similarity of distinct blocks may reduce the accuracy [4]. The proposed algorithms in this paper can be used for modeling the degree of similarity of synteny blocks in genome-wide comparison of multiple species. The rest of this paper is organized as follows. Section 2 describes the algorithm for finding the HCIS of two sequences. Section 3 illustrates the algorithm for solving the HCWIS problem of two sequences. Section 4 presents the algorithm for solving the k-HCIS problem.

2 Algorithm for the HCIS Problem of Two Sequences

Formally, let $A = \langle a_1, a_2, ..., a_m\rangle$ and $B = \langle b_1, b_2, ..., b_n\rangle$ be two number sequences, where $m > n$. Define $weight(a_i, b_j)$ as the weight of a matched pair $a_i = b_j$. Without loss of generality, we assume that the weight of each matched pair is positive. A common increasing subsequence is a subsequence $\langle a_{i_1} = b_{j_1}, a_{i_2} = b_{j_2}, ..., a_{i_l} = b_{j_l}\rangle$, where $i_1 < i_2 < ... < i_l$, $j_1 < j_2 < ... < j_l$, and $a_{i_k} < a_{i_{k+1}}$ for all $1 \leq k < l$. A heaviest common increasing subsequence is a common increasing subsequence with maximum total weight. That is, the sum of $weight(a_{i_1}, b_{j_1})$, $weight(a_{i_2}, b_{j_2})$, ..., and $weight(a_{i_l}, b_{j_l})$ is maximum among all common increasing subsequences. For example, suppose $A = \langle 1, 4, 6, 2, 3, 4\rangle$, $B = \langle 1, 2, 3, 4, 6\rangle$, and $weight(a_i, b_j) = a_i$. Then $\langle 1, 2, 3, 4\rangle$ and $\langle 1, 4, 6\rangle$ are both common increasing subsequences, but $\{1, 4, 6\}$ (with total weight 11) is heavier than the other. Note that if each matched pair has the same unit weight, this problem is equivalent to the LCIS problem. In this paper, we will interchangeably use a_i or $A[i]$ for representing the i-th number in one sequence.

Suppose $C = \langle c_1, c_2, ..., c_{l-1}, c_l\rangle$ is the heaviest common increasing subsequence of A and B. For each c_k in C (where $1 \leq k \leq l$), we denote I_k^A and I_k^B as the indices in sequence A and B corresponding to c_k, respectively (see Figure 1). Clearly, C is also the HCIS for prefix $A_{I_l^A} = \langle a_1, a_2, ..., a_{I_l^A-1}, a_{I_l^A}\rangle$ and prefix $B_{I_l^B} = \langle b_1, b_2, ..., b_{I_l^B-1}, b_{I_l^B}\rangle$. The following lemma implies that the optimal solution C can be obtained by computing the HCIS ending at preceding position for prefixes $A_{I_l^A-1}$ and $B_{I_l^B-1}$.

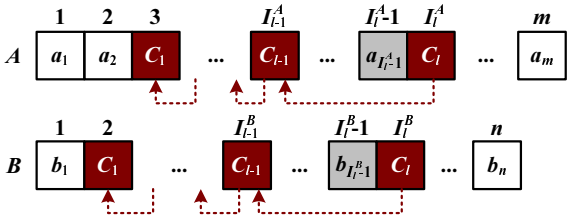

Figure 1: The prefix of optimal solution $C' = \langle c_1, c_2, ..., c_{l-1}\rangle$ is the HCIS ending at $B[I_{l-1}^B]$ for prefixes $A_{I_l^A-1}$ and $B_{I_l^B-1}$.

Lemma 1. *Let $C = \langle c_1, c_2, ..., c_l\rangle$ be the optimal solution of the HCIS problem. Then $C' = \langle c_1, c_2, ..., c_{l-1}\rangle$ is a heaviest common increasing subsequence ending at $B[I_{l-1}^B]$ for prefix*

Table 1: Summary of results of the LCIS and HCIS problems. k: number of sequences; r: number of k-tuples of positions where the k sequences match; m and n: lengths of two input sequences; α: size of the alphabet. Note that r can be as large as $O(n^k)$ and L can be up to $O(n)$.

	Previous results		Results of this paper
$k = 2$	LCIS	$O(mn)$ time and $O(mn)$ space [20] $O(\min(r \log L, nL + r) \log \log n + Sort(n))$ [5] $O((m + nL) \log \log \alpha + Sort(n))$ [3]	$O(mn)$ time and $O(n + r)$ space for HCIS and HCWIS
	LCWIS	$O(m + n \log n)$ time when $\alpha = 3$ [3]	
	HCIS	$O(mn \log n)$ time and $O(mn)$ space [18]	
$k \geq 3$	LCIS	$O(\min(r \log L \log^k r) + k * Sort(n))$ time [5] $O(\min(kr^2, r \log^{k-1} r \log \log r))$ time [3]	$O(kn^k)$ time for k-HCIS

$A_{I_l^A - 1} = \langle a_1, a_2, ..., a_{I_l^A - 1} \rangle$ *and prefix* $B_{I_l^B - 1} = \langle b_1, b_2, ..., b_{I_l^B - 1} \rangle$.

Proof. By contradiction, suppose there exists another common increasing subsequence $S = \langle s_1, s_2, ..., s_k \rangle$ for $A_{I_l^A - 1}$ and $B_{I_l^B - 1}$, which also ends at $B[I_{l-1}^B]$ and has larger weight than C'. Then the new sequence $\langle s_1, s_2, ..., s_k, c_l \rangle$ is also a common increasing subsequence for $A_{I_{c_l}^A}$ and $B_{I_{c_l}^B}$, which has larger weight than that of C. But this is a contradiction since C is optimal. Hence, C' must be a heaviest common increasing subsequence ending at $B[I_{l-1}^B]$ for $A_{I_l^A - 1}$ and $B_{I_l^B - 1}$. □

In the following, we describe a dynamic programming algorithm which incrementally computes the heaviest common increasing subsequence ending at $B[j]$ for the i-th prefix of $A = \langle a_1, a_2, ..., a_i \rangle$ and j-th prefix of $B = \langle b_1, b_2, ..., b_j \rangle$. The correctness of this algorithm is elaborated in Section 2.2. An array $HCIS[j]$ is used for storing the optimal solution ending at $B[j]$, for all $1 \leq j \leq n$. Let $HCIS[j].weight$ be the total weight of HCIS ending at $B[j]$ and $HCIS[j].number$ be the matched number $A[i] = B[j]$ for some i and j. We also define $HCIS[j].parent$ as the previous index of the HCIS ending at $B[j]$ and denote $HCIS[j].timestamp$ as the updated timestamp of this entry. Initially, $HCIS[i].weight$, $HCIS[i].parent$, and $HCIS[j].timestamp$ are set to zero, null, and zero, respectively. In order for backtracting the entire HCIS sequence, we will create a linked list $LIST[j]$ for storing all optimal solutions containing $B[j]$ but having different parent indices.

2.1 Main Algorithm

This algorithm iteratively scans all prefixes of A and B. The pseudo code of our algorithm is given in AlgorithmHCIS. Within the inner loop, the following cases are considered by our algorithm.

Case 1: $A[i] \neq B[j]$ **and** $A[i] < B[j]$**.** Since this pair of numbers is different and $A[i] < B[j]$ is not increasing, $A[i]$ will not be appended to the HCIS ending at $B[j]$ in subsequent iterations. Thus, no actions will be taken for this case.

Case 2: $A[i] \neq B[j]$ **and** $A[i] > B[j]$**.** Although this pair of numbers is different, the HCIS ending at $B[j]$ may be extended to $A[i]$ later, if we find $A[i] = B[k]$ in the k-th iteration for some $k > j$. Therefore, we maintain a variable called *maxweight* for storing the maximum weight of HCIS ending at $B[j]$ that can be possibly extended to $A[i]$. Another variable *parent* will store the index j representing the source of the *maxweight*.

Case 3: $A[i] = B[j]$**.** This pair of numbers is the same. We will compare the current $HCIS[j].weight$ with the value of *maxweight* plus $weight(A[i], B[j]))$. If the new weight is larger than the current value of $HCIS[j].weight$, $HCIS[j].weight$ will be updated to the new weight, and $HCIS[j].parent$ will be updated to the index stored in *parent*. In addition, $HCIS[j].timestamp$ will store the current value of i. Note that in order to backtract the entire sequence later, the original values of $HCIS[j]$ will be copied to a linked list called $LIST[j]$.

Algorithm: AlgorithmHCIS(A, B, m, n)
/*** Initialization ***/
1 **for** $l = 1$ to n **do**
2 $HCIS[l].weight = 0$;
$HCIS[l].parent = Null$;
$HCIS[l].timestamp = 0$;

```
/*** Main Algorithm ***/
3 for i = 1 to m do
4   maxweight = 0; parent = Null;
5   for j = 1 to n do
6     if A[i] > B[j] and HCIS[j].weight >
      maxweight
7       maxweight = HCIS[j].weight;
8       parent = j;
9     elseif A[i] = B[j] and (maxweight+
      weight(A[i], B[j]) > HCIS[j].weight)
10      Copy HCIS[j] to LIST[j];
11      HCIS[j].weight = maxweight+
        weight(A[i], B[j]);
12      HCIS[j].parent = parent;
13      HCIS[j].timestamp = i;
14      HCIS[j].number = A[i];
/*** Retrieve the maximum HCIS ***/
15 Output the maximum HCIS[k].weight;
```

At the end of this algorithm, $HCIS[j].weight$ will store the maximum weight of HCIS ending at $B[j]$. Therefore, the maximum weight in this array is the weight of the HCIS of A and B. Figure 2 illustrates an example of this algorithm for the input of $A = \langle 1, 5, 6, 2, 4, 3\rangle$ and $B = \langle 1, 6, 5, 2, 3, 4\rangle$.

2.2 Correctness

The correctness of this algorithm relies on the following lemma, which indicates that the weight of heaviest common increasing subsequences ending at any position in B can be correctly computed by our algorithm.

Lemma 2. *At the end of this algorithm, $HCIS[j].weight$ stores the weight of heaviest common increasing subsequence of A and B which ends at $B[j]$, for all $1 \le j \le n$.*

Proof. Let $A_i = \langle a_1, a_2, ..., a_i\rangle$ be the i-th prefix of A. The proof proceeds by induction on all prefixes A_i, which corresponds to the i-th iteration in Line 3 in AlgorithmHCIS. For $i = 1$, the heaviest common increasing subsequence of A_1 and B is simply the matched pair a_1 and b_j (if existed), which will be found at Line 9 in AlgorithmHCIS. It is easy to see that at the end of the first iteration of outer loop, $HCIS[j].weight$ will store the weight $a_1 = b_j$ for all such b_j, and other $HCIS.weight[j]$ will remain unchanged.

Suppose the claims holds for $i = k - 1$. That is, the algorithm can obtain the weight of HCIS (ending at all $B[j]$) for A_{k-1} and B after the $(k-1)$-th iteration. Now consider $i = k$. If a_k does not match any number in B, the optimal solutions (ending at all $B[j]$) for A_k and B are the same as those for A_{k-1} and B. Line 9 ensures that $HCIS[j].weight$ will not be updated in the k-th iteration. On the other hand, suppose a_k matches some number b_j in B. If a_k and b_j are part of the optimal solution $\langle c_1, c_2, ..., c_l\rangle$ which ends at $B[j]$ (i.e., $a_k = b_j = c_l$), the current value in $HCIS[j].weight$ should be updated. By Lemma 1, we know $\langle c_1, c_2, ..., c_{l-1}\rangle$ is the optimal solution of prefixes A_{k-1} and B_{j-1}. By induction, we have obtained the heaviest common increasing subsequences ending at all positions for A_{k-1} and B in the $(k-1)$-th iteration. Therefore, in the k-th iteration, we will retrieve the weight of $\langle c_1, c_2, ..., c_{l-1}\rangle$ in Lines 6-8 and later in Lines 9-14, we will obtain the maximum weight ending at $B[j]$. □

2.3 Backtracting

The entire heaviest common increasing subsequence can be retrieved through the link of $HCIS[j].parent$ and comparison of timestamps in the linked list $LIST[j]$. For each $HCIS[j]$, a linked list $LIST[j]$ may be created if there are multiple numbers in sequence A identical to some $B[j]$. For example, the number 4 in $A = \langle 1, 4, 5, 2, 3, 4\rangle$ and $B = \langle 1, 2, 3, 4, 5\rangle$. At the end of the main program, we can append each $HCIS[j]$ to the corresponding linked list $LIST[j]$. Note that each node in the $LIST[j]$ stands for different HCIS solutions containing $B[j]$ but having distinct parent indices. Following the above example, suppose $\langle 1, 2, 3, 4\rangle$ and $\langle 1, 4, 5\rangle$ are the optimal HCIS solutions ending at $B[4]$ and $B[5]$, respectively. Thus, $LIST[4]$ will contain two nodes with different $HCIS[4].parent$, whereas one points to $B[3]$ and the other points to $B[1]$.

Each node on the same linked list has distinct timestamps. Note that the timestamps of nodes on one HCIS solution are monotonically increasing. Given the ending node of one HCIS solution, we can backtract all the preceding nodes corresponding to this solution through the parent link and timestamps. Suppose $HCIS[max]$ is the the element with maximum weight in the $HCIS$ array. The linked list $LIST[HCIS[max].parent]$ will be retrieved. Then, we will look for the node in the list with the timestamp equal or smaller than that of the current node. If there are multiple nodes with smaller time stamps, the maximum of them is chosen. A pseudo code of the backtracting algorithm is described below.

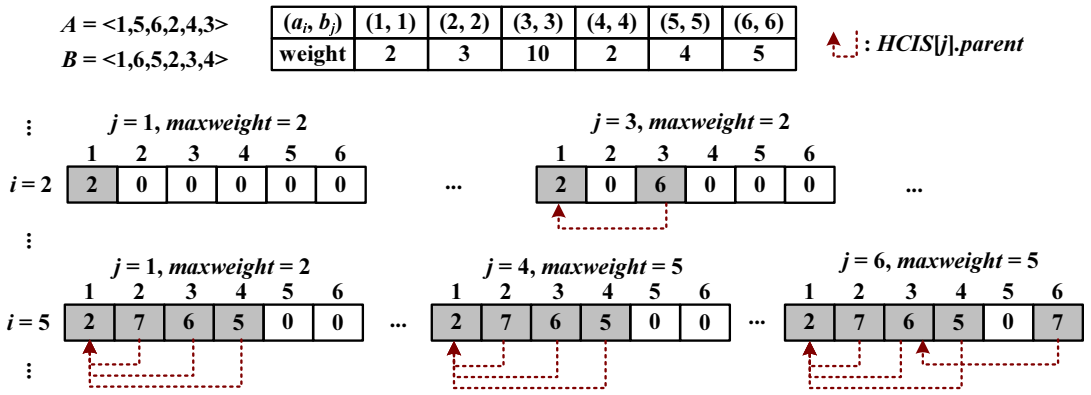

Figure 2: The values of $HCIS[j].weight$ for $i = 2$ and $i = 5$ in AlgorithmHCIS.

Algorithm: BACKTRACTING($HCIS, LIST, max$)
1 Append each $HCIS[j]$ to $LIST[j]$;
2 $Temp = HCIS[max]$;
3 **while** $Temp.parent \neq null$ **do**
4 print $Temp.number$;
5 $index = Temp.parent$;
6 Find node *parent* in list $LIST[index]$ having maximum time stamp and satisfy $parent.timestamp \leq Temp.timestamp$
7 $Temp = parent$
8 print $Temp.number$;

2.4 Analysis

The time complexity of AlgorithmHCIS is clearly $O(mn)$. The backtacting process takes $O(n + r)$. Therefore, the overall running time is bounded on $O(mn)$. The storage of the $HCIS[j]$ array takes $O(n)$ space. However, in order to backtract the entire sequence of the optimal solution, we have to use extra space for those linked lists, which takes $O(r)$. As a consequence, the space complexity of this algorithm is $O(n + r)$.

3 Heaviest Common Weakly Increasing Subsequence

This section studies the problem for finding the HCWIS of two number sequences. The input of the HCWIS problem consists of two sequences and a weight function for each pair of matched numbers. Compared with the strictly increasing requirement of the LCIS and HCIS problems, the HCWIS problem asks for a monotonically increasing (non-decreasing) subsequence (e.g., $\langle 1, 2, 2, 4\rangle$) with maximum total weight. The HCWIS problem is a generalization of the LCWIS problem [3]. Suppose $C = \langle c_1, c_2, ..., c_{l-1}, c_l\rangle$ is the optimal solution of A and B for the HCWIS problem. Denote I_k^A and I_k^B as the indices in sequence A and B corresponding to some c_k in C, respectively. Using the same arguments in Lemma 1, we have the following similar lemma for the HCWIS problem.

Lemma 3. *Let $C = \langle c_1, c_2, ..., c_l\rangle$ be the optimal solution of the HCIS problem. Then $C' = \langle c_1, c_2, ..., c_{l-1}\rangle$ is a heaviest common weakly increasing subsequence ending at $B[I_{l-1}^B]$ for prefix $A_{I_l^A-1} = \langle a_1, a_2, ..., a_{I_l^A-1}\rangle$ and prefix $B_{I_l^B-1} = \langle b_1, b_2, ..., b_{I_l^B-1}\rangle$.*

Consequently, the optimal solution of HCWIS can be computed by generalizing the AlgorithmHCIS. Note that when discovering a pair of matched numbers, this pair can be used either as the prefix of an optimal solution or as the end of an optimal solution. Therefore, the algorithm has to be revised for distinguishing these two cases. The pseudo code of the revised algorithm is given in AlgorithmHCWIS. The original variables *maxweight* and *parent* may still be updated within one iteration, but the updated values can only be used in the subsequent iteration instead of current iteration. We used additional variables *premaxweight* and *preparent* for storing the total weight and parent index which can be the best prefix at this iteration, respectively. This setting ensures that either possibility of a matched number (a prefix or an ending position) will be processed correctly.

Algorithm: ALGORITHMHCWIS(A, B, m, n)
...
for $i = 1$ to m **do**

```
    maxweight = 0; parent = Null;
    for j = 1 to n do
      premaxweight = maxweight;
      preparent = parent;
      if A[i] ≥ B[j] and HCIS[j].weight >
      maxweight
        maxweight = HCIS[j].weight;
        parent = j;
      if A[i] = B[j] and (premaxweight+
      weight(A[i], B[j])) > HCIS[j].weight
        Copy HCIS[j] to linked list LIST[j];
        HCIS[j].weight = premaxweight+
        weight(A[i], B[j]);
        HCIS[j].parent = preparent;
...
```

Clearly, this algorithm also runs in $O(mn)$ time and $O(n + r)$ space. The entire sequence can be retrieved by the similar backtracting process described in Section 2.3.

4 Algorithm for the HCIS Problem of k Sequences

In this section, we study the problem (called k-HCIS) for finding the HCIS of k number sequences, where $k \geq 3$. Let S_1, S_2, ..., and S_k be the k sequences and assume each sequence contains n numbers. Define $weight(S_1[i], S_2[j], ..., S_k[k])$ as the weight of each k-tuple of positions at which the k sequences match. Similarly, the k-HCIS problem asks for a common increasing subsequence of S_1, S_2, ..., and S_k with maximum total weight. Since the LCIS problem of k sequences is NP-hard [5, 13] and is a special case of the k-HCIS problem when setting unit weight of each matched k-tuple of positions, we have the following theorem.

Theorem 1. *The k-HCIS problem is NP-hard.*

When k is small, we present a faster algorithm than previous methods for k-LCIS problem. Suppose $C = \langle c_1, c_2, ..., c_{l-1}, c_l \rangle$ is the optimal solution for the k-HCIS problem. Using similar arguments in Lemma 1, the optimal solution of the k-HCIS problem can be obtained by computing the HCIS of some prefixes of these k sequences.

Lemma 4. *Let $C = \langle c_1, c_2, ..., c_l \rangle$ be the optimal solution of the k-HCIS problem. Then $C' = \langle c_1, c_2, ..., c_{l-1} \rangle$ is a heaviest common increasing subsequence ending at $S_k[I^k_{l-1}]$ for some prefixes of $S_1, S_2, ..., S_{k-1}$.*

Therefore, the AlgorithmHCIS can be generalized for solving the k-HCIS problem. We perform a similar dynamic programming strategy for computing the HCIS ending at each position of S_k for all prefixes of S_1, S_2, ..., and S_k. A pseudo code of this algorithm is give in Algorithm-k-HCIS. Within the algorithm, the array $HCIS[j]$ keeps the maximum weight of HCIS ending at $S_k[j]$ for some prefixes of S_1, S_2, ..., and S_{k-1}. All prefixes are scanned by a k-level loop. Let l_1, l_2, ..., and l_k be the indices within loop iterations. Note that we only have to enter an inner loop l_i if $S_1[l_1] = S_2[l_2] = ... = S_i[l_i]$. However, compared with AlgorithmHCIS, the increasing property of all k sequences has to be maintained by additional steps. The $HCIS[j].pos[p]$ (where $1 \leq p \leq k-1$) will be used to store the index in the S_p sequence contributing to the HCIS ending at $S_k[j]$. When searching for the possible prefix to be appended, $HCIS[j].pos[k]$ will be further investigated to ensure that the prefix along with the appended number is increasing in all sequences. The remaining parts are similar with previous algorithm. The maximum weight of possible preceding HCIS that can be appended by $S_{k-1}[l_{k-1}]$ is recorded. Furthermore, if we find $S_{k-1}[l_{k-1}] = S_k[l_k]$ in the subsequent iteration, $HCIS[j].weight$ is updated only if the new weight is larger than the current value.

Algorithm: ALGORITHM-k-HCIS(S_1, S_2, ..., S_k)

```
...
for l_1 = 1 to n do
  for l_2 = 1 to n do
    if (S_1[l_1] = S_2[l_2])
      ...
      for l_{k-1} = 1 to n do
        if (S_{k-2}[l_{k-2}] = S_{k-1}[l_{k-1}])
          for l_k = 1 to n do
            if S_{k-1}[l_{k-1}] > S_k[l_k] and
            HCIS[l_k].weight > maxweight and
            l_1 > HCIS[l_k].pos[1] and l_2 > ... and
            l_{k-1} > HCIS[l_k].pos[k − 1]
              maxweight = HCIS[l_k].weight;
              parent = l_k;
              ...
            elseif (S_{k-1}[l_{k-1}] = S_k[l_k] and
              (maxweight + weight(S_1[l_1], ..., S_k[l_k])
              > HCIS[j].weight))
              ...
```

The backtracting process of this algorithm can be extended from the algorithm in Section 2.3. The time complexity of entire algorithm is clearly

$O(kn^k)$. However, we need addition $O(nk)$-space for the $HCIS[j].pos[k]$. Furthermore, in order for backtracting the entire sequence of the optimal solution, the linked lists should be created for all k-tuples of positions where k sequences match. Therefore, the space complexity of this algorithm is $O(nk + r)$.

5 Conclusion

In this paper, we studied algorithms for the HCIS, HCWIS, and k-HCIS problems, which are generalizations of the LCIS, LCWIS, and k-LCIS problems, respectively. We described an algorithm with $O(mn)$ running time and with $O(n+r)$ space for the HCIS problem, where m and n are the lengths of two sequences. We also solved the HCWIS problem in $O(mn)$ time and $O(n + r)$ space. For the k-HCIS problem, we presented an $O(kn^k)$-time algorithm which runs faster than previous algorithms for the k-LCIS problem.

ACKNOWLEDGEMENT

We thank Chien-Chun Sung for providing his master thesis for reference. We are grateful for the valuable comments provided by Maw-Shang Chang and Kun-Mao Chao. Yao-Ting Huang was supported in part by NSC grant 97-2221-E-194-051 from the National Science Council, Taiwan.

References

[1] Bergroth, L. Hakonen, H., and Raita, T. A survey of longest common subsequence algorithms, *Proc. 7th International Symposium on String Processing Information Retrieval*, 39-48, 2000.

[2] Bespamyatnikh, S. and Segal, M. Enumerating longest increasing subsequences and patience sorting, *Information Processing Letters*, 76:7-11, 2000.

[3] Brodal, G.S., Kaligosi, K., Katriel, I., and Kutz, M. Faster algorithms for computing longest common increasing subsequences, *Proc. 17th Annual Symposium on Combinatorial Pattern Matching*, 330-341, 2006.

[4] Chan, H.L., Lam, T.W., Sung, W.K. Wong, W.H., Yiu, S.M., and Fan, X. The mutated subsequence problem and locating conserved genes, *Bioinformatics*, 21, 2005.

[5] Chan, W. T., Zhang, Y., Fung, P.Y., Ye, D., and Zhu, H. Efficient algorithms for finding a longest common increasing subsequence, *Journal of Combinatorial Optimization*, 13:277-288, 2007.

[6] Chvatal, V., Klarner, D.A., and Knuth, D.E. Selected combinatorial research problems, Technical Report, STAN-CS-72-292, Computer Science Department, Stanford University, 1972.

[7] Cormen, T.H., Leiserson, C.E., Rivest, R.L., and Stein, C. *Introduction to algorithms*, The MIT Press, 2001.

[8] Delcher, A.L., *et al*. Alignment of whole genomes. *Nucleic Acids Resesearch*, 27:2369-2376, 1999.

[9] Delcher, A.L., Phillippy, A., Carlton, J., and Salzberg, S.L. Fast algorithms for large-scale genome alignment and comparison, *Nucleic Acids Resesearch*, 30:2478-2483, 2002.

[10] Fredman, M.L. On computing the length of longest increasing subsequences, *Discrete Math.*, 11:29-35, 1975.

[11] Jacobson, G. and Vo, K.P. Heaviest increasing/common subsequence problems, *Proc. 3rd Annual Symposium on Combinatorial Pattern Matching*, 52-66, 1992.

[12] Knuth, D.E. Sorting and Searching, *The Art of Computer Programming, vol. 3*, Addison-Wesley, 1973.

[13] Maier, D. The complexity of some problems on subsequences and supersequences, *J. ACM*, 25(2):322-336, 1978.

[14] Masek, W.J. and Paterson, M.S. A faster algorithm computing string edit distances, *J. Comput. System Sci.*, 20(1):18-31, 1980.

[15] Pevzner, P. *Computational Molecular Biology - An Algorithmic Approach*, The MIT Press, 2000.

[16] Pevzner, P. and Tesler, G. Genome rearrangements in mammalian evolution: Lessons from human and mouse genomic sequences, *Genome Research*, 13:13-26, 2003.

[17] Schensted, C. Longest increasing and decreasing subsequences, *Canad. J. Math.*, 13:179-191, 1961.

[18] Sung, C.-C. Efficient algorithms for some heaviest subsequence problems, *Master Thesis*, National Taiwan University, 2007.

[19] Tseng, C.-T., Yang, C.-B., and Ann, H.-S. Minimal height and sequence constrained longest increasing subsequence, *Proc. of International Computer Symposium*, 2:3-8, 2008.

[20] Yang, I.-H., Huang, C.-P., and Chao, K.-M. A fast algorithm for computing a longest common increasing subsequence, *Information Processing Letters*, 249-253, 2005.